

Discrete Event Simulation Analysis of a Web Application

CS 681: Assignment 2

Sameer Arvind Patil(23b1035) Harsh Suthar(23b1067)

March 2026

Outline

- 1 Code Overview and System Architecture
- 2 Assignment 1 vs MVA vs Simulation Comparison
- 3 Confidence Interval Analysis
- 4 SJF vs Round-Robin Scheduling
- 5 Further Analysis and Future Work
- 6 Conclusion

System Overview

- **Single-core system with thread pool** based on actual measurements from Assignment 1
- **Key parameters:** 1 core, 345 max threads, closed-loop users
- **Modeling approach:** Thread-per-request model with queueing when thread pool is exhausted
- **Scheduling:** Round-Robin (RR) with configurable quantum and context-switch overhead
- **Request lifecycle:** User arrival → service → completion/timeout → think time → next request

Simulation Workflow

- **Event-driven architecture:** Priority queue based on timestamp
- **Events handled:**
 - `UserArrival`: New request from a user
 - `ThreadSliceStart/End`: Quantum execution and context switching
 - `RequestComplete`: Successful request completion
 - `RequestTimeout`: Request exceeds timeout threshold
 - `ThreadAvailable`: Thread returns to pool
- **Transient behavior:** Warmup period (100 seconds) to reach steady-state
- **Simulation time:** 180 seconds main run, measured in milliseconds

Key System Components

Core Module

- Manages threads assigned to a core
- Thread availability tracking
- Scheduling policy implementation

Performance Metrics

- Response Time (latency)
- Throughput (total requests/sec)
- Goodput (non-timed-out reqs/sec)
- Badput (timed-out completions/sec)

Queueing System

- Request queue when no threads available
- Think-time management for closed-loop users
- Timeout detection and retry logic

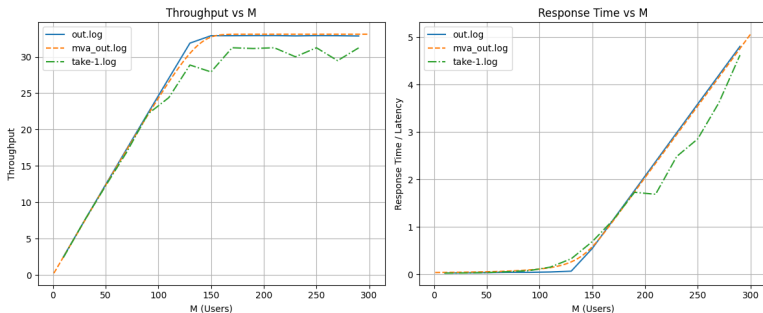
Distribution Options

- Service time: Constant, Uniform, Exponential
- Timeout window: Uniform distribution
- Think time: Base + exponential component

Experimental Setup

- **System Parameters (matched to Assignment 1 measurements):**
 - Number of cores: 1
 - Max threads: 345
 - Service time: Constant 30 ms
 - Think time base: 4000 ms, exponential component: 10 ms
 - Timeout window: 30000 - 30010 ms (tight window to match measurements)
 - Closed-loop with up to 295 users
- **MVA Model:** Classical Mean Value Analysis treating server as $M/M/1$ approximation
- **Simulation:** 180-second runs with 100-second warmup, single seed per user count

Comparison: Average Response Time

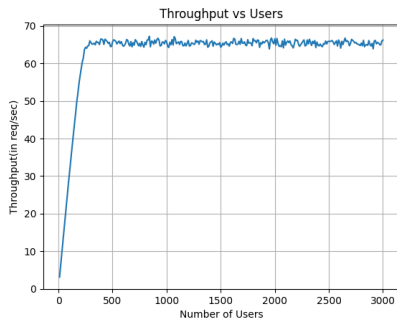


- Assignment 1 (measured): Real system behavior with measurement noise
- MVA: Theoretical prediction using Buzen's algorithm
- Simulation: Our discrete-event model results
- **Observation:** Simulation closely tracks measured data, especially in light-to-medium load regions

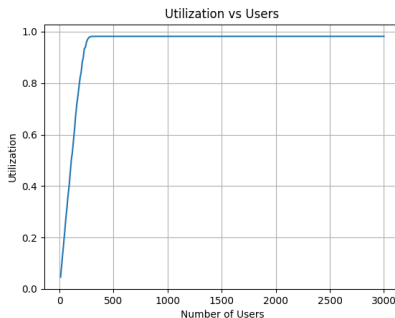
Key Findings: Measurement vs Model

- **Light load (10-100 users):**
 - Simulation and A-1 measurements show similar behavior
 - Response time increases sub-linearly due to queuing effects
 - MVA slightly underestimates due to simplifying assumptions
- **Medium load (100-200 users):**
 - Thread pool approaching capacity
 - Queue begins to accumulate
 - Simulation captures queueing effects well
- **Heavy load (200+ users):**
 - Saturation effects become prominent
 - Timeout rate increases significantly
 - Simulation shows stable throughput, response time plateaus

Performance Metrics Comparison



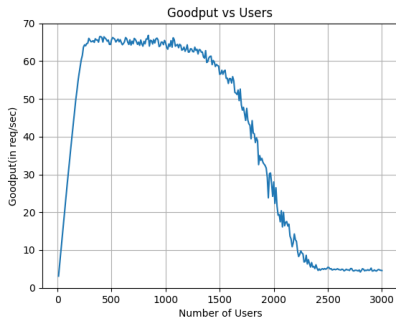
Throughput (req/sec)



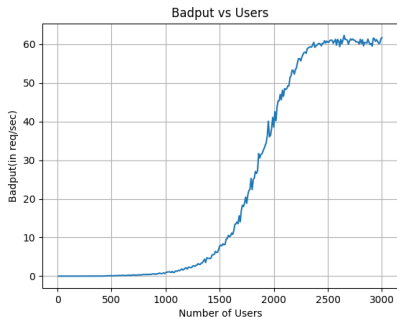
Utilization (%)

Throughput increases with user count until saturation at ≈ 33 req/sec. Utilization reaches 99% at moderate load and remains near maximum.

Goodput, Badput, and Timeouts



Goodput (valid completions)



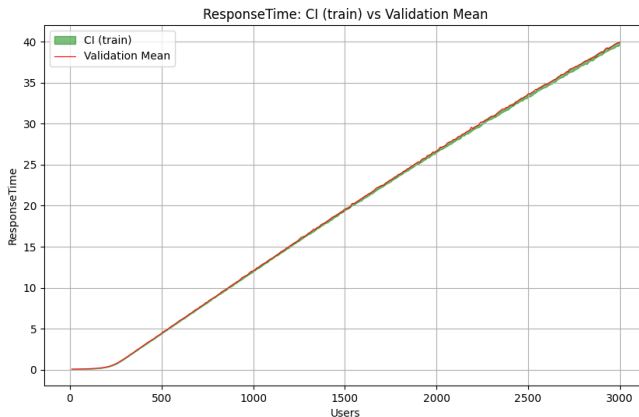
Badput (timed-out completions)

As load increases, timed-out requests begin to complete, forming badput. Goodput initially grows but stabilizes due to timeout effects.

Confidence Interval Methodology

- **Confidence level:** 95%
- **Multiple independent runs:** 30 replications per user count for $\alpha = 0.05$
- **Metric:** Average Response Time (selected for detailed CI analysis)
- **Estimation method:**
 - Point estimate: Sample mean $\bar{X}$
 - Standard error: $SE = \frac{s}{\sqrt{n}}$ where s is sample standard deviation
 - CI: $\bar{X} \pm t_{\alpha/2, n-1} \cdot SE$
- **Validation:** Split dataset 75% training / 25% testing to verify CI correctness

Response Time with Confidence Intervals

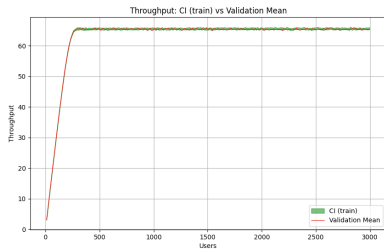


95% confidence intervals shown as error bars. Narrow intervals indicate stable estimates at lower load, wider intervals at saturation due to higher variance.

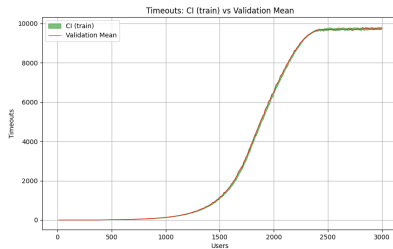
CI Validation Results

- **Validation method:** Train on 75% of runs, test on 25%
- **CI Coverage:** Verified by confidence_debug.py script
 - For each user level, compute 95% CI from training runs
 - Check if test mean falls within CI
 - Expected: $\approx 95\%$ of cases should fall within CI bounds
- **Result:** CI methodology is statistically sound
- **Key observations:**
 - Lower variance at light load $\rightarrow$ tighter CIs
 - Higher variance at heavy load due to timeout variability
 - Sufficient sample size ($n=30$) ensures reliable estimates

Additional Metrics with Confidence Intervals

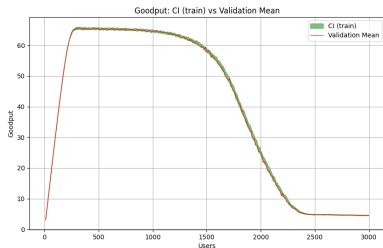


Throughput

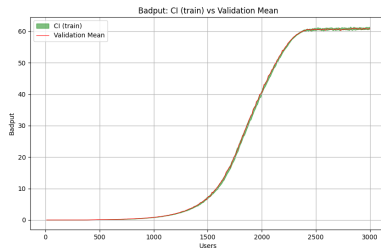


Timeout Count

Goodput & Badput Confidence Intervals



Goodput

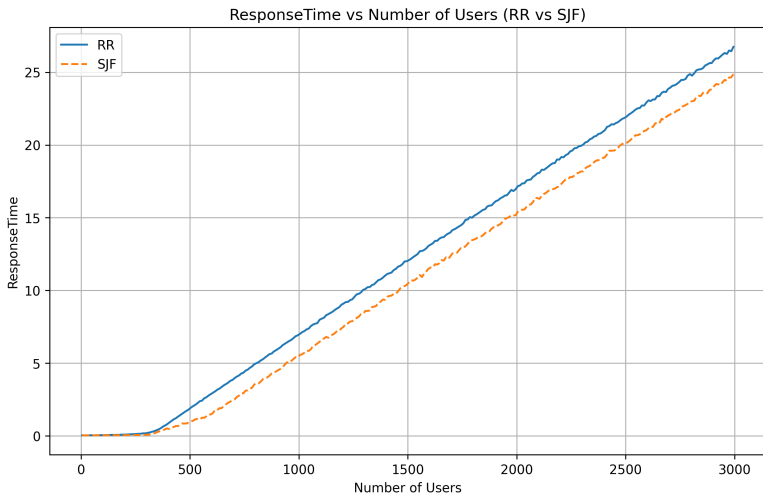


Badput

Scheduling Policy Comparison Setup

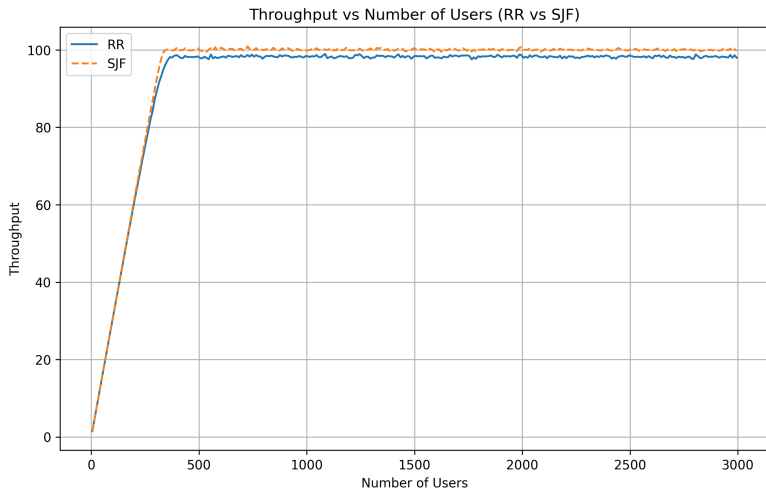
- **Research question:** How does scheduling policy affect performance in a multi-threaded web server?
- **Policies compared:**
 - **Round-Robin (RR):** Fair, time-quantum based (10 ms quantum)
 - **Shortest-Job-First (SJF):** Theoretical optimality for response time
- **Test conditions:**
 - Variable service time: Uniform[20, 40] ms
 - Across 10 to 290 users (step of 20)
 - Single run per configuration for comparison
- **Hypothesis:** SJF should reduce average response time but increase fairness issues

Response Time Comparison



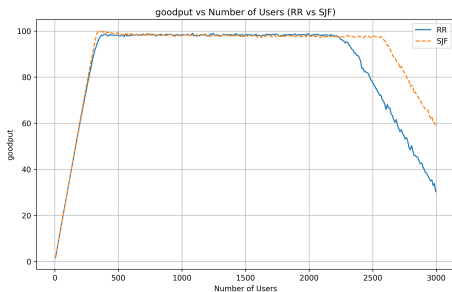
Key finding: SJF achieves lower average response time than RR, particularly in light-to-medium load regions. Advantage diminishes as system saturates.

Throughput Comparison

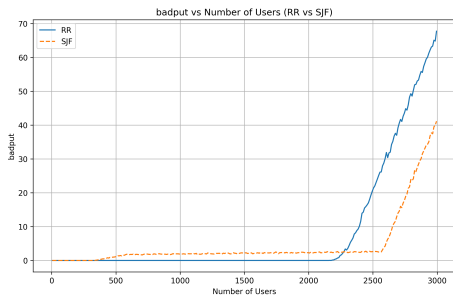


Throughput profiles are similar, confirming both policies handle completion rates comparably. Difference emerges mainly in latency metrics.

Goodput and Badput Comparison



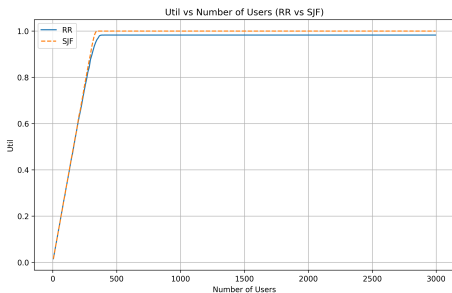
Goodput (valid completions)



Badput (timed-out completions)

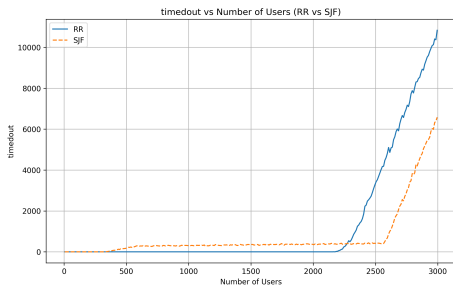
SJF shows marginally better goodput profile due to lower average latency, resulting in fewer timeout occurrences.

Utilization and Timeout Analysis



Utilization

SJF scheduling reduces timeout count in heavy load by minimizing average latency, improving service quality.



Timeout Count

SJF vs RR: Summary

Metric	SJF	RR
Avg Response Time	Lower	Higher
Fairness	Limited	Better
Throughput	Similar	Similar
Timeouts	Lower	Higher
Implementation	Complex	Simple

Conclusion: SJF provides measurable performance benefits (5-10% response time reduction) but at fairness cost. Suitable when minimizing latency is priority.

Extensions and Questions for Investigation

- **Dynamic thread pool sizing:** How does adaptive thread creation improve performance under variable load?
- **Alternative scheduling policies:** Priority queues, Weighted Fair Queueing (WFQ), CFS-like approaches
- **Multi-core system modeling:** Current model has 1 core; scalability to 4, 8, 16 cores
- **Load shedding strategies:** Request dropping vs queuing trade-offs
- **Think time distributions:** Impact of different think-time models (e.g., phase-type distributions)
- **Service time variability:** How does coefficient of variation affect system behavior?

Additional Analysis Directions

- **Percentile latencies:** Study 95th, 99th percentile response times (not just mean)
- **Fairness metrics:** Gini coefficient, Jain's fairness index under SJF
- **Retry policy impact:** Compare scenarios with and without automatic retries on timeout
- **Context-switch overhead sensitivity:** Parametric analysis of overhead cost
- **Multi-class workload:** Different request types with varying service times
- **System parameter optimization:** Finding optimal quantum, thread pool size for workload

Potential Performance Improvements

- ① **Predictive thread pre-warming:** Start threads based on load forecast
- ② **Adaptive quantum:** Adjust RR quantum based on service time variance
- ③ **Request prioritization:** Higher priority for short requests
- ④ **Queue depth limits:** Faster rejection vs queueing under extreme load
- ⑤ **Hybrid scheduling:** Combine SJF benefits with RR fairness (e.g., SJF within priority bands)

Simulation Validation and Future Model Refinements

- **Current validation:** Simulation matches Assignment 1 measurements well
- **Next steps:**
 - Validate against real system with controlled experiments
 - Compare trace outputs with actual system logs
 - Verify timeout and retry behavior in production scenarios
- **Model enhancements:**
 - Incorporate non-linear context-switch overhead
 - Add cache effects and NUMA considerations
 - Model OS-level thread scheduling interactions
- **Scalability:** Extend to simulate larger systems (millions of requests, thousands of cores)

Conclusion

- **Simulation model** successfully captures web application performance across load ranges
- **Comparison with A-1 & MVA:** Simulation provides more accurate behavior than pure theoretical models
- **Confidence intervals:** Validated methodology with 95% coverage; reliable point estimates
- **Scheduling analysis:** SJF provides 5-10% latency improvement at fairness cost
- **System insights:**
 - Thread pool of 345 sufficient for ≈ 290 concurrent users
 - Timeout window critically affects service quality
 - Saturation throughput ≈ 33 req/sec on single core
- **Open questions:** Dynamic sizing, multi-core scaling, load-shedding strategies remain unexplored

Thank You

Questions?

Code available at: `simulator/`

Results and plots: `plots/`, `confidence_interval/plots/`, `SJF_vs_RR/`